Android, MySQL, PHP, & JSON 4 | Register and Login PHP Scripts

Posted By trav on May 30, 2013 | 0 comments

This entry is part 20 of 22 in the series Android-Intermediate

Creating PHP Login, Registration, and Comment Scripts!

In this Android Remote Databases mini series we'll start setting up our php scripts to register a user, login, write a message, and display some JSON data. We will also be building some temporary front-end php forms to test out our scripts, but eventually we will develop the front-end code within our Android app.

Follow the Remote Databases for Android Series!

- Part 1: Android Remote Databases and Servers Overview
- Part 2: Local Xampp Server and MySQL Database
- Part 3: Working with a Remote Server and MySQL
- **Part 4: Creating Login and Registration PHP Scripts**
- Part 5: Developing the Android Application
- Part 6: JSON Parsing and Android ListView Design

PHP Web Service Script

For Android Video Tutorial

Part A: Learn the Basics of PHP

Part B: Learn Android 2.20 B –

Creating a PHP Web Service for Apps

Part C: Coming soon…

Developing PHP Script to Interact with our Database

The core concept of developing an Android app that connects with a remote database is to have the front-end programming done within the Android application, then send the information to a PHP script to be processed, the script will then interact with the database, and last the information will be returned back to the app. In this tutorial we will be building the PHP scripts to interact with the database.

Note- These scripts will not be too secure since anyone can interact with them, but it should demonstrate the basics of this mini series.

Let's update our PHP 'webservice' folder to contain some new PHP scripts. In the previous tutorial we setup our **"config.inc.php"** and **"index.php"**, but let's add some new files.

Add the following php files:
-register.php
-login.php
-comments.php

-addcomment.php

Step 1: Creating a New User PHP Script

Before we can login to our account, we need to have an account. We will be creating a temporary front-end form to test our registration script.

The main focus of this script will be:
-To receive the username and password data passed from the front-end form.
-To check if the username already exist.
-If it does exist return an error message.
-If it doesn't exist, create a new user within our MySQL database, and return a success message.

For the purpose of a through tutorial series, I will assume you have little to no knowledge of PHP, so I'll briefly cover the basics while we setup the registration page. PHP is used for 2 main purposes, creating dynamic content and processing information. To understand how php processes information, it is almost easier to explain by example. First, let's create a very simple front-end registration form, open up the register.php file and add the following code:

register.php v.01

```
<?php

/*
Our "config.inc.php" file connects to database every time we include or require
it within a php script.  Since we want this script to add a new user to our db,
we will be talking with our database, and therefore,
let's require the connection to happen:
*/
    require("config.inc.php");
?>

<h1>Register</h1>
<form action="register.php" method="post">
    Username:<br />
    <input type="text" name="username" value="" />
    <br /><br />
    Password:<br />
    <input type="password" name="password" value="" />
    <br /><br />
    <input type="submit" value="Register New User" />
</form>
```

Simple enough, we include our "config.inc.php" script to connect to our database. Then we create a simple html form that has a username and password inputs and a submit button. Once the submit button is pressed, the form will get the information from the two inputs and pass that data to the url path defined by the form's "action" attribute. In this case it will reload the page, but we will be sending data to the reloaded page, make since so far?

The way that we are sending this information is by posting, 99% of the time you will want to use "post" as your method. We also don't wan't to load up the html form again if there is posted data, instead we will want to load up a success or failure message. Lets add an if statement to dynamically determine what to display, a html form or a message.

We don't need to make this look pretty, because we will be creating the official front-end form from within our Android application. However, if you were building something like a social network, you probably would want to have a web interface as well as a mobile app, if this is the case, you may want swagify the form with some nice CSS.

register.php v.02

```
<?php

/*
Our "config.inc.php" file connects to database every time we include or require
it within a php script.  Since we want this script to add a new user to our db,
we will be talking with our database, and therefore,
let's require the connection to happen:
*/
    require("config.inc.php");

//if posted data is not empty
 if(!empty($_POST))
    {
        echo 'message';
    }
 else
    {
```

```
    ?>
    <h1>Register</h1>
    <form action="register.php" method="post">
        Username:<br />
        <input type="text" name="username" value="" />
        <br /><br />
        Password:<br />
        <input type="password" name="password" value="" />
        <br /><br />
        <input type="submit" value="Register New User" />
    </form>
    <?php
    }

?>
```

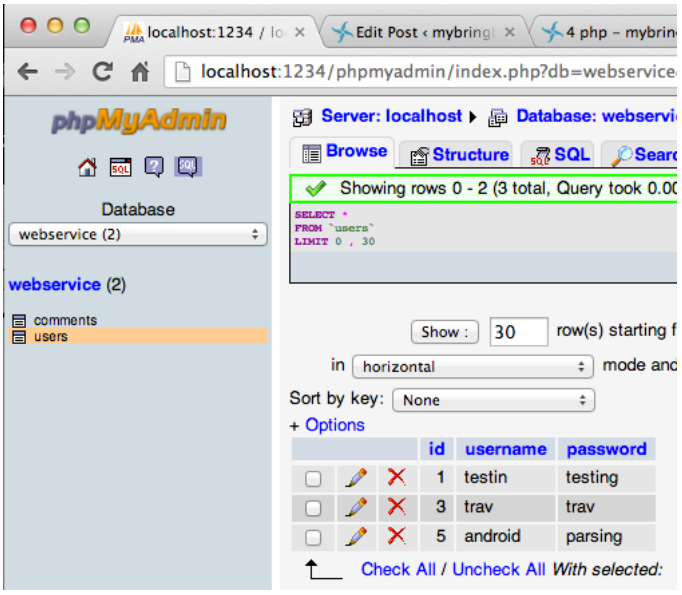
Go ahead and test it out. If you are using XAMPP, make sure the control panel is running both Apache and MySQL. Any time you type "../register.php" you will see a form, but if you fill out the form and post data, it will not include a form. The goal is to use our Android app to post data to this url and have it return a JSON message for us to parse. If this url had a form, it may affect the parsing we will do from within our mobile app. Now that, you have the basics down, let's see our final script.

register.php v.02

```
<?php

/*
Our "config.inc.php" file connects to database every time we include or require
it within a php script.  Since we want this script to add a new user to our db,
we will be talking with our database, and therefore,
let's require the connection to happen:
*/
require("config.inc.php");

//if posted data is not empty
if (!empty($_POST)) {
    //If the username or password is empty when the user submits
    //the form, the page will die.
    //Using die isn't a very good practice, you may want to look into
    //displaying an error message within the form instead.
    //We could also do front-end form validation from within our Android App,
    //but it is good to have a have the back-end code do a double check.
    if (empty($_POST['username']) || empty($_POST['password'])) {


        // Create some data that will be the JSON response
        $response["success"] = 0;
        $response["message"] = "Please Enter Both a Username and Password.";

        //die will kill the page and not execute any code below, it will also
        //display the parameter... in this case the JSON data our Android
        //app will parse
        die(json_encode($response));
    }

    //if the page hasn't died, we will check with our database to see if there is
    //already a user with the username specificed in the form.  ":user" is just
    //a blank variable that we will change before we execute the query.  We
    //do it this way to increase security, and defend against sql injections
    $query        = " SELECT 1 FROM users WHERE username = :user";
    //now lets update what :user should be
    $query_params = array(
        ':user' => $_POST['username']
    );

    //Now let's make run the query:
    try {
        // These two statements run the query against your database table.
        $stmt   = $db->prepare($query);
        $result = $stmt->execute($query_params);
    }
    catch (PDOException $ex) {
        // For testing, you could use a die and message.
        //die("Failed to run query: " . $ex->getMessage());

        //or just use this use this one to product JSON data:
        $response["success"] = 0;
        $response["message"] = "Database Error1. Please Try Again!";
        die(json_encode($response));

    }

    //fetch is an array of returned data.  If any data is returned,
    //we know that the username is already in use, so we murder our
    //page
    $row = $stmt->fetch();
    if ($row) {
        // For testing, you could use a die and message.
        //die("This username is already in use");
```

```php

        //You could comment out the above die and use this one:
        $response["success"] = 0;
        $response["message"] = "I'm sorry, this username is already in use";
        die(json_encode($response));
    }

    //If we have made it here without dying, then we are in the clear to
    //create a new user.  Let's setup our new query to create a user.
    //Again, to protect against sql injects, user tokens such as :user and :pass
    $query = "INSERT INTO users ( username, password ) VALUES ( :user, :pass ) ";

    //Again, we need to update our tokens with the actual data:
    $query_params = array(
        ':user' => $_POST['username'],
        ':pass' => $_POST['password']
    );

    //time to run our query, and create the user
    try {
        $stmt   = $db->prepare($query);
        $result = $stmt->execute($query_params);
    }
    catch (PDOException $ex) {
        // For testing, you could use a die and message.
        //die("Failed to run query: " . $ex->getMessage());

        //or just use this use this one:
        $response["success"] = 0;
        $response["message"] = "Database Error2. Please Try Again!";
        die(json_encode($response));
    }

    //If we have made it this far without dying, we have successfully added
    //a new user to our database.  We could do a few things here, such as
    //redirect to the login page.  Instead we are going to echo out some
    //json data that will be read by the Android application, which will login
    //the user (or redirect to a different activity, I'm not sure yet..)
    $response["success"] = 1;
    $response["message"] = "Username Successfully Added!";
    echo json_encode($response);

    //for a php webservice you could do a simple redirect and die.
    //header("Location: login.php");
    //die("Redirecting to login.php");


} else {
?>
    <h1>Register</h1>
    <form action="register.php" method="post">
        Username:<br />
        <input type="text" name="username" value="" />
        <br /><br />
        Password:<br />
        <input type="password" name="password" value="" />
        <br /><br />
        <input type="submit" value="Register New User" />
    </form>
    <?php
}

?>
```

I believe that I've commented the code enough for you to get the idea of what is going on. We basically just go through a variety of tests, and if any of those test fail, then we will kill the page and display a JSON error message. If none of the test fail we do not kill the page and the user is added to the database. Go ahead and test it out. If you go to your phpmyadmin page you will see the new users within our user table:



Note: As you can see in the picture I had registered 5 users, but I deleted users with the id of 2 and 4. In a production site, we would want to encrypt the user's password with MD5 at least, if not something more secure, but in this example we don't do any encrypting.

I hope you now get the basic idea of our php scripts. I'll just provide the code and add some comments for each of the following scripts.

Step 2:

Developing a Login Script

login.php

```php
<?php

//load and connect to MySQL database stuff
require("config.inc.php");

if (!empty($_POST)) {
    //gets user's info based off of a username.
    $query = "
            SELECT
                id,
                username,
                password
            FROM users
            WHERE
                username = :username
        ";

    $query_params = array(
        ':username' => $_POST['username']
    );

    try {
        $stmt   = $db->prepare($query);
        $result = $stmt->execute($query_params);
    }
    catch (PDOException $ex) {
        // For testing, you could use a die and message.
        //die("Failed to run query: " . $ex->getMessage());

        //or just use this use this one to product JSON data:
        $response["success"] = 0;
        $response["message"] = "Database Error1. Please Try Again!";
        die(json_encode($response));

    }

    //This will be the variable to determine whether or not the user's information is correct.
    //we initialize it as false.
    $validated_info = false;

    //fetching all the rows from the query
    $row = $stmt->fetch();
```

```
    if ($row) {
        //if we encrypted the password, we would unencrypt it here, but in our case we just
        //compare the two passwords
        if ($_POST['password'] === $row['password']) {
            $login_ok = true;
        }
    }

    // If the user logged in successfully, then we send them to the private members-only page
    // Otherwise, we display a login failed message and show the login form again
    if ($login_ok) {
        $response["success"] = 1;
        $response["message"] = "Login successful!";
        die(json_encode($response));
    } else {
        $response["success"] = 0;
        $response["message"] = "Invalid Credentials!";
        die(json_encode($response));
    }
} else {
?>
        <h1>Login</h1>
        <form action="login.php" method="post">
            Username:<br />
            <input type="text" name="username" placeholder="username" />
            <br /><br />
            Password:<br />
            <input type="password" name="password" placeholder="password" value="" />
            <br /><br />
            <input type="submit" value="Login" />
        </form>
        <a href="register.php">Register</a>
    <?php
}

?>
```

Step 3: Writing a New Comment

addcomment.php

```
<?php

//load and connect to MySQL database stuff
require("config.inc.php");

if (!empty($_POST)) {
    //initial query
    $query = "INSERT INTO comments ( username, title, message ) VALUES ( :user, :title, :message ) ";

    //Update query
    $query_params = array(
        ':user' => $_POST['username'],
        ':title' => $_POST['title'],
        ':message' => $_POST['message']
    );

    //execute query
    try {
        $stmt   = $db->prepare($query);
        $result = $stmt->execute($query_params);
    }
    catch (PDOException $ex) {
        // For testing, you could use a die and message.
        //die("Failed to run query: " . $ex->getMessage());

        //or just use this use this one:
        $response["success"] = 0;
        $response["message"] = "Database Error. Couldn't add post!";
        die(json_encode($response));
    }

    $response["success"] = 1;
    $response["message"] = "Username Successfully Added!";
    echo json_encode($response);

} else {
?>
        <h1>Add Comment</h1>
        <form action="addcomment.php" method="post">
            Username:<br />
            <input type="text" name="username" placeholder="username" />
            <br /><br />
            Title:<br />
            <input type="text" name="title" placeholder="post title" />
            <br /><br />
```

```
            Message:<br />
            <input type="text" name="message" placeholder="post message" />
            <br /><br />
            <input type="submit" value="Add Comment" />
        </form>
    <?php
}

?>
```

Step 4: Displaying JSON Data – Recent Comments

comments.php

```
<?php

/*
Our "config.inc.php" file connects to database every time we include or require
it within a php script.  Since we want this script to add a new user to our db,
we will be talking with our database, and therefore,
let's require the connection to happen:
*/
require("config.inc.php");

//initial query
$query = "Select * FROM comments";

//execute query
try {
    $stmt   = $db->prepare($query);
    $result = $stmt->execute($query_params);
}
catch (PDOException $ex) {
    $response["success"] = 0;
    $response["message"] = "Database Error!";
    die(json_encode($response));
}

// Finally, we can retrieve all of the found rows into an array using fetchAll
$rows = $stmt->fetchAll();


if ($rows) {
    $response["success"] = 1;
    $response["message"] = "Post Available!";
    $response["posts"]   = array();

    foreach ($rows as $row) {
        $post             = array();
        $post["username"] = $row["username"];
        $post["title"]    = $row["title"];
        $post["message"]  = $row["message"];


        //update our repsonse JSON data
        array_push($response["posts"], $post);
    }

    // echoing JSON response
    echo json_encode($response);


} else {
    $response["success"] = 0;
    $response["message"] = "No Post Available!";
    die(json_encode($response));
}

?>
```

Awesome! I think we are done with most of our php scripts. Even though most of them are pretty rudimentary, they should do the trick for our application, but I just want to note how easily our system could get hacked if someone finds out where we are hosting our scripts, but you can learn about building a secure system another day.

Let's add some minor adjustments to our 'comments' database and scripts. We want to add a unique id so that we can make edits to posts, display the oldest, or newest posts, etc. You may even want to add more fields to the comments data to meet all of your needs, but I'm just going to keep this tutorial simple.

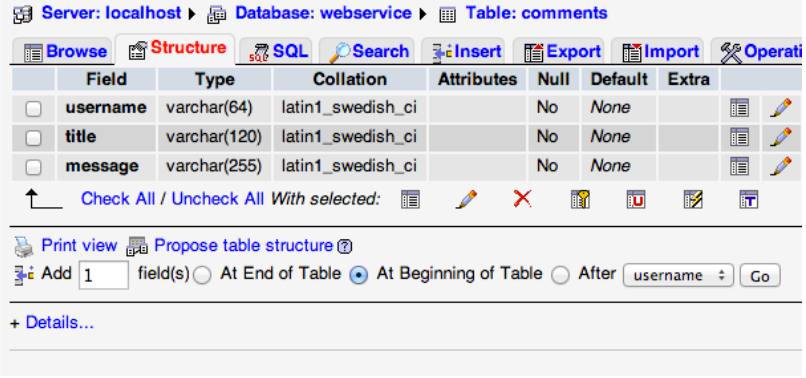
Step 5: Final Edits

Modify our comments.php to include a post id.

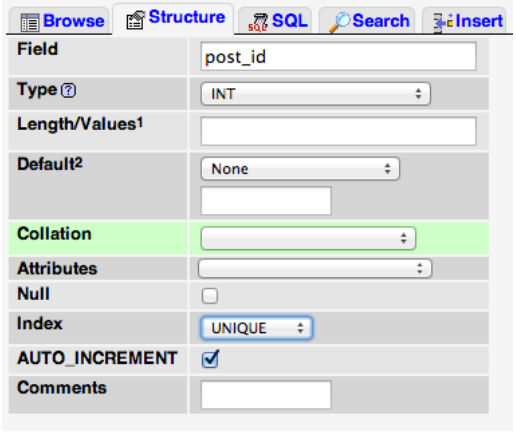
comments.php

```
....

 foreach ($rows as $row) {
        $post             = array();

        //this line is new:
        $post["post_id"]  = $row["post_id"];

        $post["username"] = $row["username"];
        $post["title"]    = $row["title"];
        $post["message"]  = $row["message"];


        //update our repsonse JSON data
        array_push($response["posts"], $post);
    }

...
```

Now, let's update our database, go to phpMyAdmin, and select our 'comments' table from our 'webservice' database, and then click on the "Structure" tab. Add one new row at the beginning of the table called "post_id".



The field set "post_id" will be unique and auto-increment.



Everything should now be updated and you can check out your "addcomment.php" script to add some new comments to the database, and then you can go to the "comments.php" url and you will see something like this:

```
{"success":1,"message":"Post Available!","posts":
[{"post_id":"1","username":"trav","title":"Testing this title","message":"Here is my
message"},{"post_id":"2","username":"trav","title":"Title 2","message":"This is going
to be awesome!"},{"post_id":"3","username":"bored guy","title":"I'm really
bored","message":"... like really bored."}]}
```

This is kind of hard to browse through, but if you use an online JSON formatter, you can copy and paste the code and it will format it to look like this:

```
{ "message" : "Post Available!",
  "posts" : [ { "message" : "Here is my message",
        "post_id" : "1",
        "title" : "Testing this title",
        "username" : "trav"
      },
      { "message" : "This is going to be awesome!",
        "post_id" : "2",
        "title" : "Title 2",
        "username" : "trav"
      },
      { "message" : "... like really bored.",
        "post_id" : "3",
        "title" : "I'm really bored",
        "username" : "bored guy"
```

```
        }
    ],
  "success" : 1
}
```

Everything is looking pretty good so far. Our JSON data is being displayed in a somewhat organized fashion, we are able to register a user, check if a user's login credentials are legit, add a comment, and display all of the current comments.

Wrapping up!

Even though you may not know what JSON data is yet, we have successfully displayed information from our database in an easy to parse way. We have also finished up our very simple php scripts for our web service. In the next Remote Databases for Android tutorial, we will start creating our Android application. We will probably be done with this mini-series in 1 or 2 tutorials, so make sure you finish strong! See ya there.

Download the Source!

Download the Source code for this tutorial:

mybringback's Android Remote SQL Source Code 4

Previous Lesson Donate Next Lesson

Author: trav

I'm just an average guy that love programming.

Submit a Comment

Your email address will not be published. Required fields are marked *

Name *

Email *

Website

CAPTCHA Image

CAPTCHA Code *

Comment

You may use these HTML tags and attributes: <a href="" title=""> <abbr title=""> <acronym title=""> <b> <blockquote cite=""> <cite> <code> <del datetime=""> <i> <q cite=""> <strike>

Submit Comment

Tutorials